Range Queries and Segment Trees

Chapters 9 and 28 of *Competitive Programmer's Handbook* (A. Laaksonen)

AIL7002 · Data Structures and Algorithms (refresher)

IIT Delhi

Range queries

A range query asks for a value computed over a subarray $[a, b]$:

- $\text{sum}_q(a, b)$ – sum of the values in $[a, b]$
- $\min_q(a, b)$, $\max_q(a, b)$ – smallest / largest value

Range queries

A range query asks for a value computed over a subarray $[a, b]$:

- $\text{sum}_q(a, b)$ – sum of the values in $[a, b]$
- $\min_q(a, b)$, $\max_q(a, b)$ – smallest / largest value

The obvious loop is $O(n)$ per query, so $O(nq)$ for q queries – too slow when $n, q \approx 10^5$.

From the book (p. 83):

```
int sum(int a, int b) {
    int s = 0;
    for (int i = a; i <= b; i++) {
        s += array[i];
    }
    return s;
}
```



range [3, 6]

```
sum_q(3,6) = 14
min_q(3,6) = 1
max_q(3,6) = 6
```

Roadmap: which structure for which job?

Structure	Query	Update	Build	Supports
Prefix sum array	$O(1)$	– (rebuild $O(n)$)	$O(n)$	sums (invertible operations)
Sparse table	$O(1)$	–	$O(n \log n)$	min, max, gcd (overlap-safe)
Binary indexed tree	$O(\log n)$	point, $O(\log n)$	$O(n \log n)$	sums
Segment tree	$O(\log n)$	point, $O(\log n)$	$O(n)$	any associative operation
Lazy segment tree	$O(\log n)$	range, $O(\log n)$	$O(n)$	range increase/assign + range sum, min, ...

Learning outcomes

By the end of this lecture you should be able to

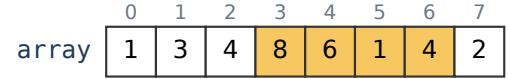
1. answer static sum queries with prefix sums (1D, 2D) and static minimum queries with a sparse table;
2. implement a binary indexed tree and explain why k&–k is the length of the range stored at k;
3. implement a segment tree bottom-up and top-down, and adapt it to min, max, gcd, …;
4. explain lazy propagation and use it for range updates with range queries;
5. recognise when a dynamic, persistent or two-dimensional segment tree is needed.

Static array queries

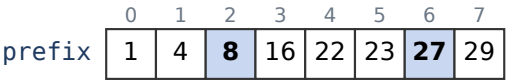
Prefix sums

Array never changes $\Rightarrow$ precompute once in $O(n)$, answer every sum query in $O(1)$:

$$\text{sum}_q(a, b) = \text{sum}_q(0, b) - \text{sum}_q(0, a-1), \qquad \text{sum}_q(0, -1) = 0.$$



8+6+1+4 = 19



27 − 8 = 19

```
sum_q(3,6) = sum_q(0,6) - sum_q(0,2)
```

Prefix sums: the code

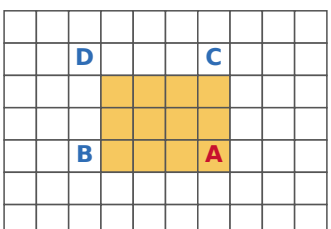
```
// p[0] = 0 and p[k+1] = sum_q(0,k)
void build() {
    p[0] = 0;
    for (int k = 0; k < n; k++) p[k+1] = p[k] + array[k];
}
int sum(int a, int b) {   // sum_q(a,b) in O(1)
    return p[b+1] - p[a];
}
```

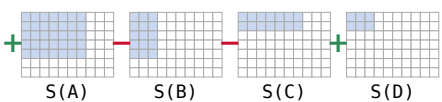
- The offset of one ($p[k+1] = \text{sum}_q(0, k)$, $p[0] = 0$) removes the special case $a = 0$.
- Works for any invertible operation (sum, xor) – not for min or max.

Two-dimensional prefix sums

$P[i][j]$ = sum of the rectangle from the upper-left corner to cell $(i-1, j-1)$; any rectangle by inclusion–exclusion:

$$\text{sum} = S(A) - S(B) - S(C) + S(D)$$





S(X) = sum of the rectangle from the upper-left corner to X

```
sum = S(A) - S(B) - S(C) + S(D)
```

Two-dimensional prefix sums: the code

Added – the book gives the formula and picture; compiled and tested against brute force:

```
// P[i][j] = sum of the rectangle with corners (0,0) and (i-1,j-1)
void build() {
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            P[i][j] = a[i-1][j-1] + P[i-1][j] + P[i][j-1] - P[i-1][j-1];
}
// sum of rows y1..y2 and columns x1..x2 (inclusive)
int sum(int y1, int x1, int y2, int x2) {
    return P[y2+1][x2+1] - P[y1][x2+1] - P[y2+1][x1] + P[y1][x1];
}
```

Why the prefix-sum trick does *not* work for min/max

For sums, prefix information is enough:

$$P[i] = \sum_{t=0}^{i} A[t] \qquad \Longrightarrow \qquad \sum_{t=l}^{r} A[t] = P[r] - P[l-1]$$

This works because addition is reversible.

For minimum, define

$$M[i] = \min_{0 \le t \le i} A[t].$$

Then

$$M[r] = \min\left(M[l-1], \min(A[l..r])\right),$$

but from this we cannot recover $\min(A[l..r])$.

Reason: the prefix minimum loses information.

Example:

$$A = [1, 5, 7], \qquad B = [1, 2, 3].$$

Both have the same prefix minima:

$$[1, 1, 1],$$

but

$$\min(A[1..2]) = 5, \qquad \min(B[1..2]) = 2.$$

Key takeaway

Prefix sums work because sum has an inverse. Range minimum/maximum does *not* have such an inverse.

Sparse table intuition: precompute answers for power-of-two intervals

Instead of storing prefix minima, we store minima for intervals of lengths

$$1, 2, 4, 8, \ldots$$

i.e., powers of two.
Define:

$$ST[k][i] = \min\{A[i], A[i+1], \ldots, A[i+2^k-1]\}.$$

So:

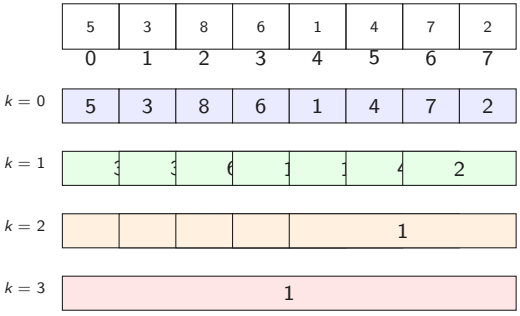
- $ST[0][i]$ stores minimum on intervals of length 1
- $ST[1][i]$ stores minimum on intervals of length 2
- $ST[2][i]$ stores minimum on intervals of length 4
- etc.

Recurrence:

$$ST[k][i] = \min\left(ST[k-1][i],\ ST[k-1][i+2^{k-1}]\right).$$

Idea: an interval of length 2^k is split into two halves of length 2^{k-1}.

Example: $A = [5, 3, 8, 6, 1, 4, 7, 2]$



Each entry stores the minimum over one interval. Preprocessing cost: $O(n \log n)$.

Why this helps

Any query interval can be represented using a small number of power-of-two blocks. For min/max, we can do even better: just two overlapping blocks suffice.

Minimum queries: the sparse table

Precompute $\min_q(a, b)$ for every range whose length is a power of two – $O(n \log n)$ values:

$$\min_q(a, b) = \min\big(\min_q(a, a + w - 1),\ \min_q(a + w, b)\big), \qquad w = \tfrac{b-a+1}{2}.$$

Minimum queries: the sparse table

Precompute $\min_q(a, b)$ for every range whose length is a power of two – $O(n \log n)$ values:

$$\min_q(a, b) = \min\big(\min_q(a, a + w - 1),\ \min_q(a + w, b)\big), \qquad w = \tfrac{b-a+1}{2}.$$

Query: let k be the largest power of two $\leq b - a + 1$ and cover $[a, b]$ by two ranges of length k, which may overlap:

$$\min_q(a, b) = \min\big(\min_q(a, a + k - 1),\ \min_q(b - k + 1, b)\big).$$

Minimum queries: the sparse table

Precompute $\min_q(a, b)$ for every range whose length is a power of two – $O(n \log n)$ values:

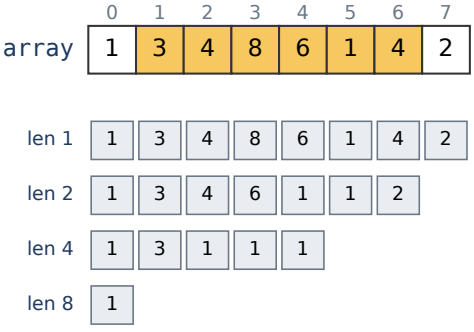
$$\min_q(a, b) = \min\big(\min_q(a, a + w - 1),\ \min_q(a + w, b)\big), \qquad w = \tfrac{b-a+1}{2}.$$

Query: let k be the largest power of two $\leq b - a + 1$ and cover $[a, b]$ by two ranges of length k, which may overlap:

$$\min_q(a, b) = \min\big(\min_q(a, a + k - 1),\ \min_q(b - k + 1, b)\big).$$

The overlap is harmless because $\min(x, x) = x$ – the same trick fails for sums.

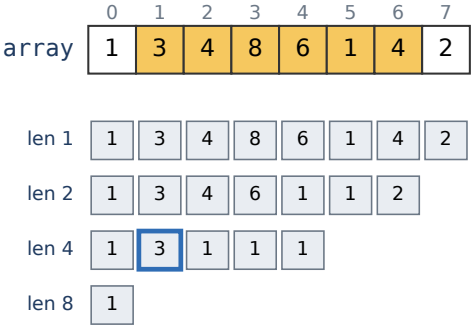
Sparse table in action: $\min_q(1, 6)$



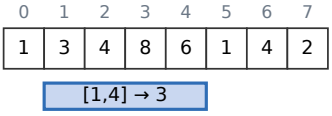
Query [1, 6]: length 6, k = 4

0	1	2	3	4	5	6	7
1	3	4	8	6	1	4	2

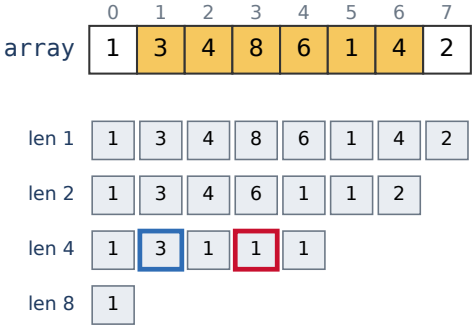
Sparse table in action: $\min_q(1, 6)$



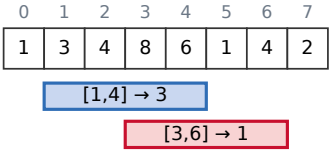
Query [1, 6]: length 6, k = 4



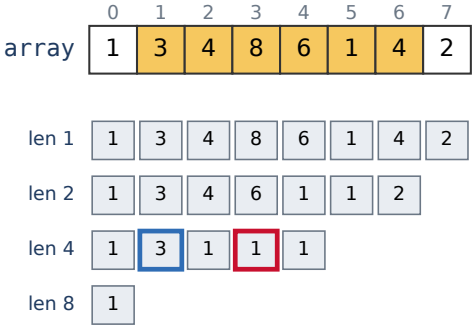
Sparse table in action: $\min_q(1, 6)$



Query [1, 6]: length 6, k = 4

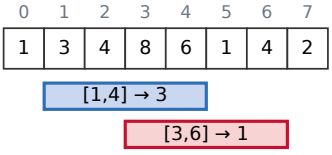


Sparse table in action: $\min_q(1, 6)$



row ‘len L’, column i = min_q(i, i+L−1)

Query [1, 6]: length 6, k = 4



min(3, 1) = 1

overlap is harmless for min

Sparse table: the code

```
// mn[j][i] = min_q(i, i+2^j-1)
void build() {
    for (int i = 0; i < n; i++) mn[0][i] = array[i];
    for (int j = 1; (1<<j) <= n; j++)
        for (int i = 0; i + (1<<j) <= n; i++)
            mn[j][i] = min(mn[j-1][i], mn[j-1][i+(1<<(j-1))]);
}
int minq(int a, int b) {
    // 2^j = the largest power of two <= b-a+1
    int j = 31 - __builtin_clz(b-a+1);
    return min(mn[j][a], mn[j][b-(1<<j)+1]);
}
```

`build`: $O(n \log n)$ time and memory; `minq`: $O(1)$; `31 - __builtin_clz(len)` $= \lfloor \log_2 len \rfloor$. Time: 2 min. A precomputed log table (lg[1] = 0, lg[i] = lg[i/2] + 1) is an alternative to ___builtin_clz.

Binary indexed tree

When the array changes

- Prefix sums break: one update can change $O(n)$ prefix sums.

When the array changes

- Prefix sums break: one update can change $O(n)$ prefix sums.
- Goal: sum of $[a, b]$ and change one value, both in $O(\log n)$.

When the array changes

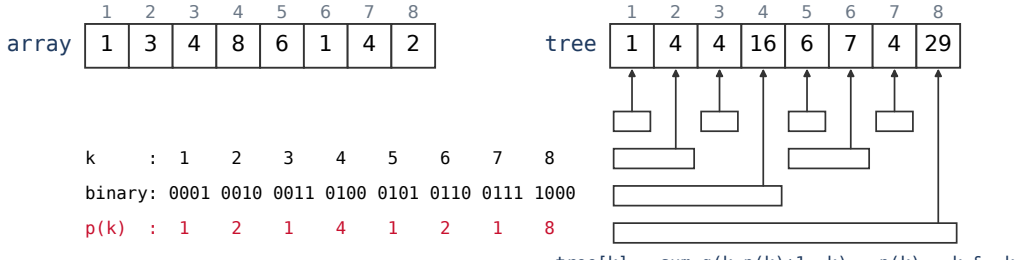
- Prefix sums break: one update can change $O(n)$ prefix sums.
- Goal: sum of $[a, b]$ and change one value, both in $O(\log n)$.
- Binary indexed tree (Fenwick tree, 1994): a *dynamic prefix-sum array*. It is 1-indexed – that makes the bit tricks work.

Binary indexed tree: what is stored

$p(k)$ = largest power of two dividing k = lowest 1-bit of k = k&–k.

$-k$ is computed in 2's complement, which preserves the rightmost 1

$$\texttt{tree}[k] = \text{sum}_q\big(k - p(k) + 1,\ k\big) \quad \text{(the } p(k) \text{ values ending at position } k\text{)}$$



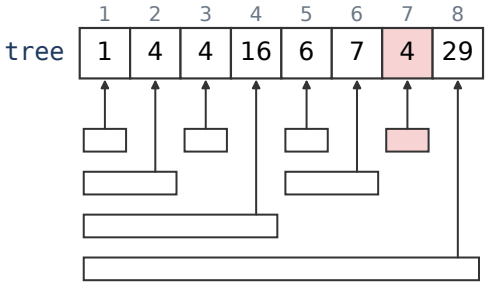
Query: $\text{sum}_q(1, k)$ in $O(\log n)$

From the book (p. 88):

```
int sum(int k) {
    int s = 0;
    while (k >= 1) {
        s += tree[k];
        k -= k&-k;
    }
    return s;
}
```

Each step clears the lowest 1-bit of k: at most $\log_2 n + 1$ steps.

$\text{sum}_q(a, b) =$
$\text{sum}_q(1, b) - \text{sum}_q(1, a - 1)$.



`k = 7 = 0111₂: s += tree[7] = 4`

`s = 4;  k -= k&-k → 6`

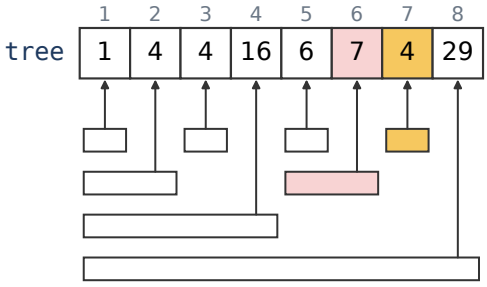
Query: $\text{sum}_q(1, k)$ in $O(\log n)$

From the book (p. 88):

```
int sum(int k) {
    int s = 0;
    while (k >= 1) {
        s += tree[k];
        k -= k&-k;
    }
    return s;
}
```

Each step clears the lowest 1-bit of k: at most $\log_2 n + 1$ steps.

$\text{sum}_q(a, b) =$
$\text{sum}_q(1, b) - \text{sum}_q(1, a - 1)$.



`k = 6 = 0110₂: s += tree[6] = 7`

`s = 11; k -= k&-k → 4`

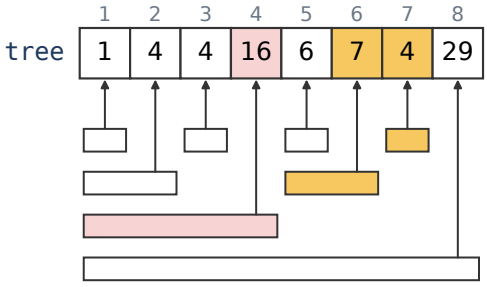
Query: $\text{sum}_q(1, k)$ in $O(\log n)$

From the book (p. 88):

```
int sum(int k) {
    int s = 0;
    while (k >= 1) {
        s += tree[k];
        k -= k&-k;
    }
    return s;
}
```

Each step clears the lowest 1-bit of k: at most $\log_2 n + 1$ steps.

$\text{sum}_q(a, b) =$
$\text{sum}_q(1, b) - \text{sum}_q(1, a - 1)$.



`k = 4 = 0100₂: s += tree[4] = 16`

`s = 27;  k -= k&-k → 0`

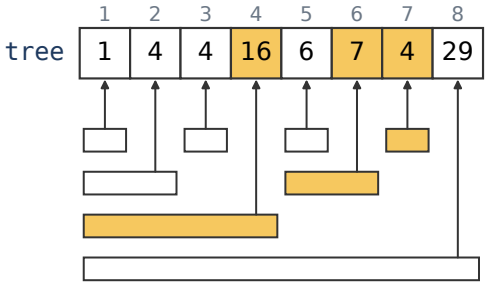
Query: $\text{sum}_q(1, k)$ in $O(\log n)$

From the book (p. 88):

```
int sum(int k) {
    int s = 0;
    while (k >= 1) {
        s += tree[k];
        k -= k&-k;
    }
    return s;
}
```

Each step clears the lowest 1-bit of k: at most $\log_2 n + 1$ steps.

$\text{sum}_q(a, b) =$
$\text{sum}_q(1, b) - \text{sum}_q(1, a - 1)$.



`k = 0 → stop`

`sum(7) = 4 + 7 + 16 = 27`

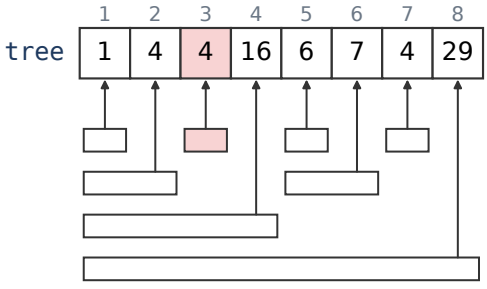
Update: add x at position k in $O(\log n)$

From the book (p. 88):

```
void add(int k, int x) {
    while (k <= n) {
        tree[k] += x;
        k += k&-k;
    }
}
```

Each step adds the lowest 1-bit: the next larger range that contains k.

Build: `add(k, array[k])` for every $k - O(n \log n)$.



$k = 3 = 0011_2$: `tree[3] += x`

`range [3,3] ∋ 3;  k += k&-k → 4`

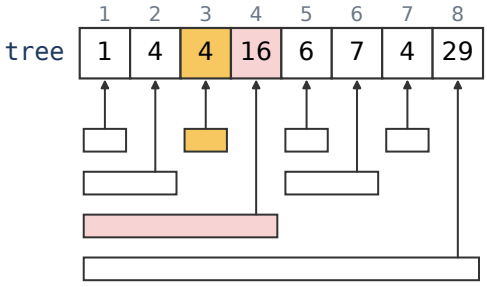
Update: add x at position k in $O(\log n)$

From the book (p. 88):

```
void add(int k, int x) {
    while (k <= n) {
        tree[k] += x;
        k += k&-k;
    }
}
```

Each step adds the lowest 1-bit: the next larger range that contains k.

Build: `add(k, array[k])` for every k – $O(n \log n)$.



`k = 4 = 0100₂: tree[4] += x`

`range [1,4] ∋ 3; k += k&-k → 8`

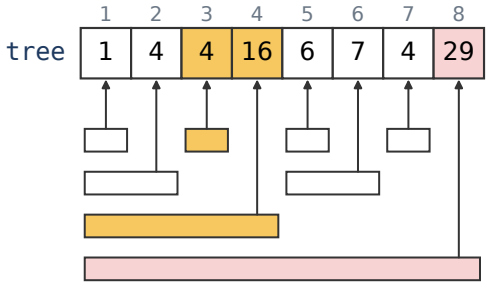
Update: add x at position k in $O(\log n)$

From the book (p. 88):

```
void add(int k, int x) {
    while (k <= n) {
        tree[k] += x;
        k += k&-k;
    }
}
```

Each step adds the lowest 1-bit: the next larger range that contains k.

Build: `add(k, array[k])` for every k – $O(n \log n)$.



`k = 8 = 1000₂: tree[8] += x`

`range [1,8] ∋ 3; k += k&-k → 16`

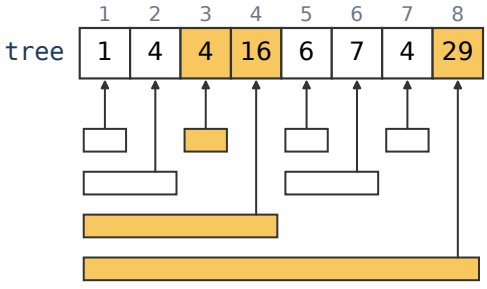
Update: add x at position k in $O(\log n)$

From the book (p. 88):

```
void add(int k, int x) {
    while (k <= n) {
        tree[k] += x;
        k += k&-k;
    }
}
```

Each step adds the lowest 1-bit: the next larger range that contains k.

Build: `add(k, array[k])` for every k – $O(n \log n)$.



`k = 16 > n → stop`

`changed: tree[3], tree[4], tree[8]`

Range updates, point queries: the difference array

	0	1	2	3	4	5	6	7
array	3	3	1	1	1	5	2	2

$diff[k] = array[k] - array[k-1]$
$\Rightarrow array[k] = \sum_{i \le k} diff[i]$.

difference	3	0	−2	0	0	4	−3	0

Adding x to $[a, b]$ changes only $diff[a]$ and $diff[b+1]$.

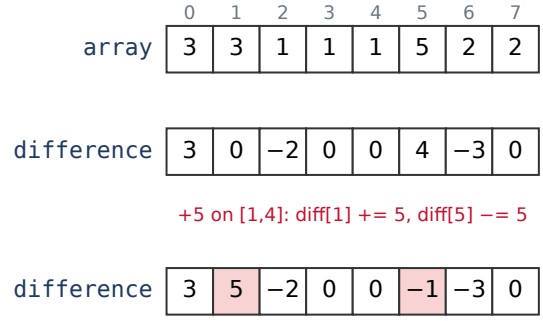
Keep *diff* in a binary indexed tree: both operations $O(\log n)$.

Range updates, point queries: the difference array

$diff[k] = array[k] - array[k-1]$
$\Rightarrow array[k] = \sum_{i \le k} diff[i]$.

Adding x to $[a, b]$ changes only $diff[a]$ and $diff[b+1]$.

Keep *diff* in a binary indexed tree: both operations $O(\log n)$.

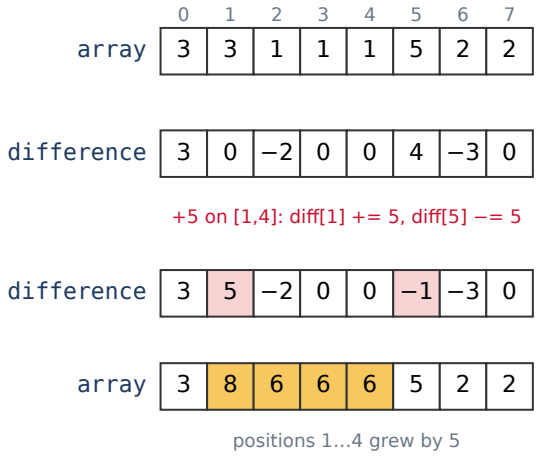


Range updates, point queries: the difference array

$diff[k] = array[k] - array[k-1]$
$\Rightarrow array[k] = \sum_{i \le k} diff[i]$.

Adding x to $[a, b]$ changes only $diff[a]$ and $diff[b+1]$.

Keep *diff* in a binary indexed tree: both operations $O(\log n)$.



Range updates with a binary indexed tree: the code

Added – compiled and tested; uses `sum` and `add` from the previous slides (1-indexed):

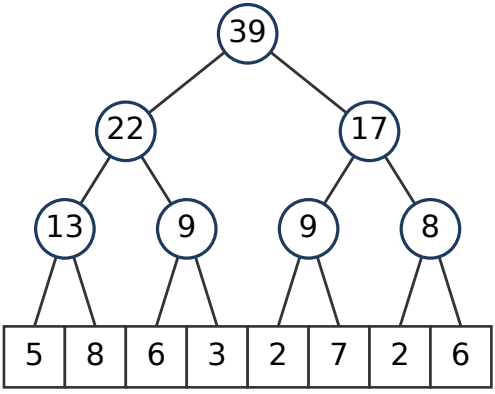
```
// tree is a binary indexed tree over the DIFFERENCE array
void increase(int a, int b, int x) {   // add x to positions a..b
    add(a, x);
    add(b+1, -x);
}
int value(int k) {                     // current value at position k
    return sum(k);
}
```

- If $b + 1 > n$, `add(b+1, -x)` does nothing – which is correct.
- Range updates *and* range queries together need lazy propagation (Chapter 28).

Segment tree

Segment tree: structure

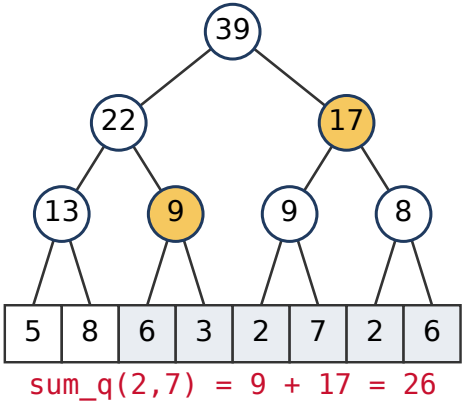
- Leaves = array elements; each internal node stores the value (here: the sum) of its range.
- n a power of two (pad with zeros); 0-based.
- Works for any operation that combines two halves: sum, min, max, gcd, and, or, xor.



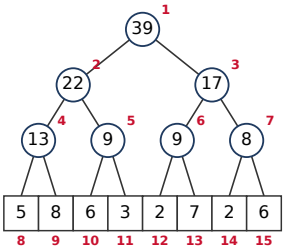
Any range = $O(\log n)$ nodes

Use nodes as high as possible: at most two per level.

$\text{sum}_q(2,7) = 6+3+2+7+2+6 = 26$ is read from just two nodes: $9 + 17$.



Storing the tree in an array



stored as an array of 2n elements

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
39	22	17	13	9	9	8	5	8	6	3	2	7	2	6

tree[1] = root

children of k: 2k and 2k+1

parent of k: ⌊k/2⌋

array values: tree[n] … tree[2n−1]

Added – tested: $O(n)$ build after copying the array to `tree[n..2n-1]`:

```
// array values are first copied to tree[n] ... tree[2n-1]
void build() {
    for (int k = n-1; k >= 1; k--) tree[k] = tree[2*k]+tree[2*k+1];
}
```

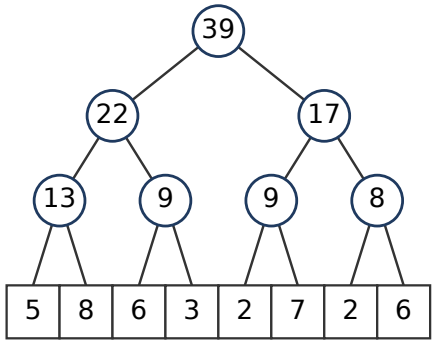
Point update = one path to the root

From the book (p. 91):

```
void add(int k, int x) {
    k += n;
    tree[k] += x;
    for (k /= 2; k >= 1; k /= 2) {
        tree[k] = tree[2*k]+tree[2*k+1];
    }
}
```

Path length $\log_2 n + 1 \Rightarrow O(\log n)$.

To assign instead of add: `tree[k] = x`.



`add(5, 3)`: before

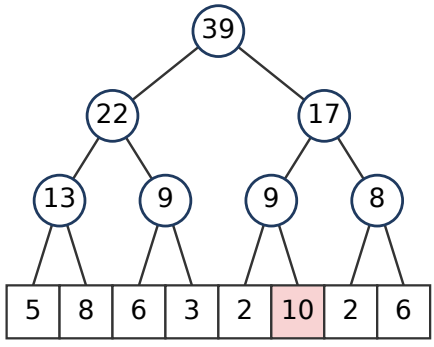
Point update = one path to the root

From the book (p. 91):

```
void add(int k, int x) {
    k += n;
    tree[k] += x;
    for (k /= 2; k >= 1; k /= 2) {
        tree[k] = tree[2*k]+tree[2*k+1];
    }
}
```

Path length $\log_2 n + 1 \Rightarrow O(\log n)$.

To assign instead of add: `tree[k] = x`.



`tree[13] += 3` → 10

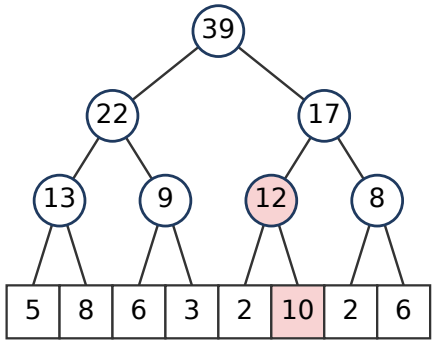
Point update = one path to the root

From the book (p. 91):

```
void add(int k, int x) {
    k += n;
    tree[k] += x;
    for (k /= 2; k >= 1; k /= 2) {
        tree[k] = tree[2*k]+tree[2*k+1];
    }
}
```

Path length $\log_2 n + 1 \Rightarrow O(\log n)$.

To assign instead of add: `tree[k] = x`.



`tree[6] = 2 + 10 = 12`

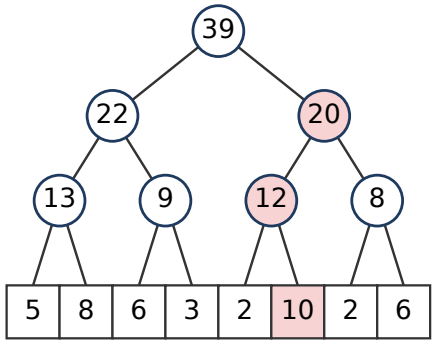
Point update = one path to the root

From the book (p. 91):

```
void add(int k, int x) {
    k += n;
    tree[k] += x;
    for (k /= 2; k >= 1; k /= 2) {
        tree[k] = tree[2*k]+tree[2*k+1];
    }
}
```

Path length $\log_2 n + 1 \Rightarrow O(\log n)$.

To assign instead of add: `tree[k] = x`.



`tree[3] = 12 + 8 = 20`

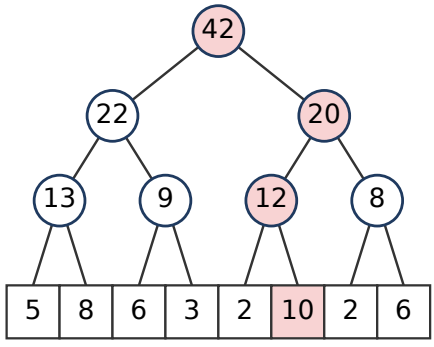
Point update = one path to the root

From the book (p. 91):

```
void add(int k, int x) {
    k += n;
    tree[k] += x;
    for (k /= 2; k >= 1; k /= 2) {
        tree[k] = tree[2*k]+tree[2*k+1];
    }
}
```

Path length $\log_2 n + 1 \Rightarrow O(\log n)$.

To assign instead of add: `tree[k] = x`.



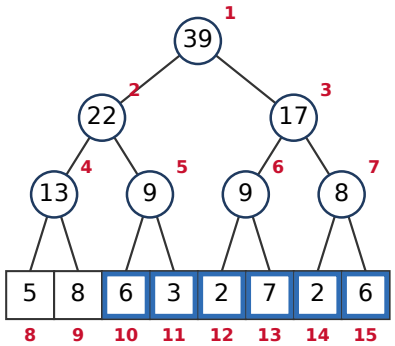
`tree[1] = 22 + 20 = 42`

Bottom-up range sum

From the book (p. 91):

```
int sum(int a, int b) {
    a += n; b += n;
    int s = 0;
    while (a <= b) {
        if (a%2 == 1) s += tree[a++];
        if (b%2 == 0) s += tree[b--];
        a /= 2; b /= 2;
    }
    return s;
}
```

Blue outline: the current range $[a, b]$. A left end at a right child (odd) cannot be covered by its parent – add it and step right; symmetrically for the right end.



a = 10, b = 15, s = 0

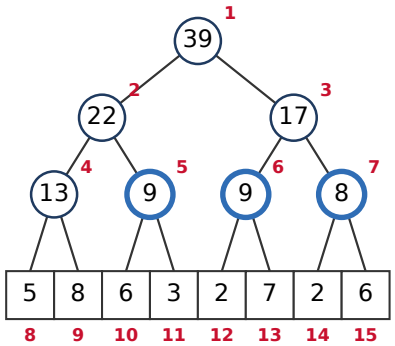
`start: a = 2+n, b = 7+n`

Bottom-up range sum

From the book (p. 91):

```
int sum(int a, int b) {
    a += n; b += n;
    int s = 0;
    while (a <= b) {
        if (a%2 == 1) s += tree[a++];
        if (b%2 == 0) s += tree[b--];
        a /= 2; b /= 2;
    }
    return s;
}
```

Blue outline: the current range $[a, b]$. A left end at a right child (odd) cannot be covered by its parent – add it and step right; symmetrically for the right end.



a = 5, b = 7, s = 0

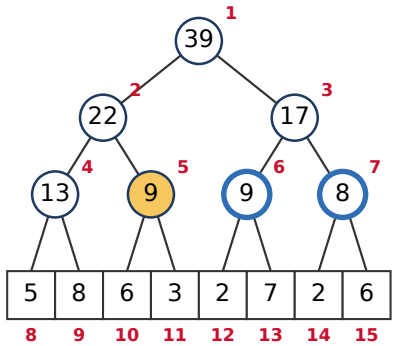
`a /= 2, b /= 2  (one level up)`

Bottom-up range sum

From the book (p. 91):

```
int sum(int a, int b) {
    a += n; b += n;
    int s = 0;
    while (a <= b) {
        if (a%2 == 1) s += tree[a++];
        if (b%2 == 0) s += tree[b--];
        a /= 2; b /= 2;
    }
    return s;
}
```

Blue outline: the current range $[a, b]$. A left end at a right child (odd) cannot be covered by its parent – add it and step right; symmetrically for the right end.



a = 6, b = 7, s = 9

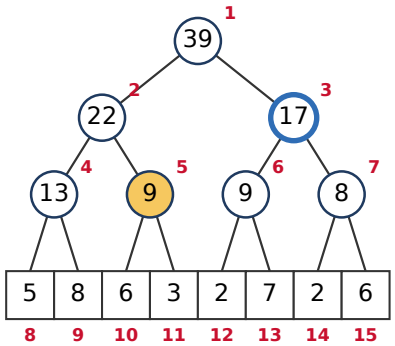
a = 5 odd → s += tree[5], a++

Bottom-up range sum

From the book (p. 91):

```
int sum(int a, int b) {
    a += n; b += n;
    int s = 0;
    while (a <= b) {
        if (a%2 == 1) s += tree[a++];
        if (b%2 == 0) s += tree[b--];
        a /= 2; b /= 2;
    }
    return s;
}
```

Blue outline: the current range $[a, b]$. A left end at a right child (odd) cannot be covered by its parent – add it and step right; symmetrically for the right end.



a = 3, b = 3, s = 9

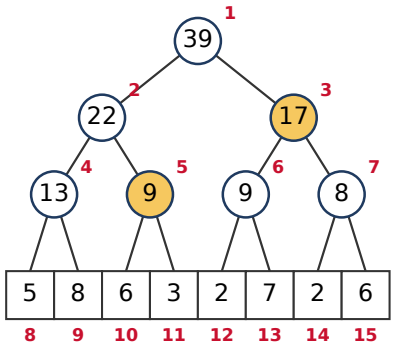
`a /= 2, b /= 2  (one level up)`

Bottom-up range sum

From the book (p. 91):

```
int sum(int a, int b) {
    a += n; b += n;
    int s = 0;
    while (a <= b) {
        if (a%2 == 1) s += tree[a++];
        if (b%2 == 0) s += tree[b--];
        a /= 2; b /= 2;
    }
    return s;
}
```

Blue outline: the current range $[a, b]$. A left end at a right child (odd) cannot be covered by its parent – add it and step right; symmetrically for the right end.



a = 4, b = 3, s = 26

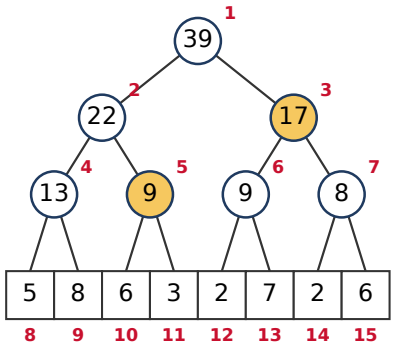
a = 3 odd → s += tree[3], a++

Bottom-up range sum

From the book (p. 91):

```
int sum(int a, int b) {
    a += n; b += n;
    int s = 0;
    while (a <= b) {
        if (a%2 == 1) s += tree[a++];
        if (b%2 == 0) s += tree[b--];
        a /= 2; b /= 2;
    }
    return s;
}
```

Blue outline: the current range $[a, b]$. A left end at a right child (odd) cannot be covered by its parent – add it and step right; symmetrically for the right end.



a = 2, b = 1, s = 26

`a > b → stop`

Other queries: min, max, gcd, xor, ...

- Works whenever a range answer can be combined from the answers of two halves.
- Min: store minima, combine with `min`, start from $+\infty$.
- In add: `tree[k] = min(tree[2*k], tree[2*k+1])`.

Added – the book says "like previously, but minima"; tested:

```
int minq(int a, int b) {
    a += n; b += n;
    int s = INT_MAX;
    while (a <= b) {
        if (a%2 == 1) s = min(s, tree[a++]);
        if (b%2 == 0) s = min(s, tree[b--]);
        a /= 2; b /= 2;
    }
    return s;
}
```

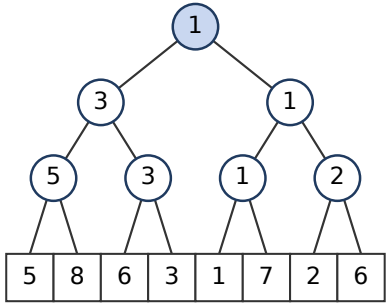
Binary search inside a segment tree

Walk down from the root, always into a child that holds the minimum: $O(\log n)$.

Added – the book describes it in words; tested:

```
// min tree: position of a smallest element
int find_min() {
    int k = 1;
    while (k < n) {
        if (tree[2*k] == tree[k]) k = 2*k;
        else k = 2*k+1;
    }
    return k-n;
}
```

Same pattern: first position with value $\geq x$, k-th one in a 0/1 array, ...



`k = 1: children 3 and 1`

min 1 is in the right child

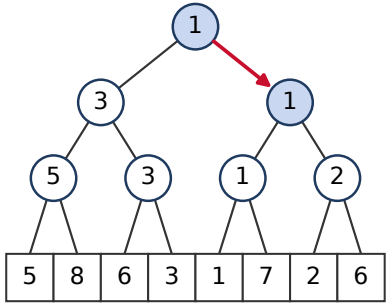
Binary search inside a segment tree

Walk down from the root, always into a child that holds the minimum: $O(\log n)$.

Added – the book describes it in words; tested:

```
// min tree: position of a smallest element
int find_min() {
    int k = 1;
    while (k < n) {
        if (tree[2*k] == tree[k]) k = 2*k;
        else k = 2*k+1;
    }
    return k-n;
}
```

Same pattern: first position with value $\geq x$, k-th one in a 0/1 array, ...



`k = 3: children 1 and 2`

min 1 is in the left child

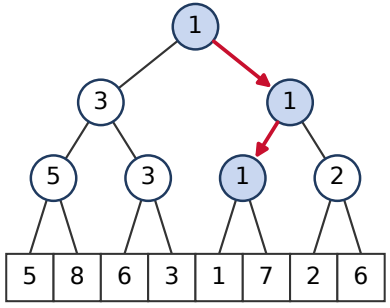
Binary search inside a segment tree

Walk down from the root, always into a child that holds the minimum: $O(\log n)$.

Added – the book describes it in words; tested:

```
// min tree: position of a smallest element
int find_min() {
    int k = 1;
    while (k < n) {
        if (tree[2*k] == tree[k]) k = 2*k;
        else k = 2*k+1;
    }
    return k-n;
}
```

Same pattern: first position with value $\geq x$, k-th one in a 0/1 array, ...



`k = 6: children 1 and 7`

min 1 is in the left child

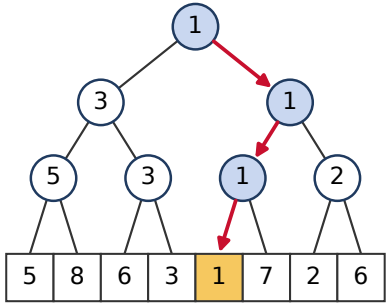
Binary search inside a segment tree

Walk down from the root, always into a child that holds the minimum: $O(\log n)$.

Added – the book describes it in words; tested:

```
// min tree: position of a smallest element
int find_min() {
    int k = 1;
    while (k < n) {
        if (tree[2*k] == tree[k]) k = 2*k;
        else k = 2*k+1;
    }
    return k-n;
}
```

Same pattern: first position with value $\geq x$, k-th one in a 0/1 array, ...



`k = 12 ≥ n → return k − n = 4`

position 4 holds the minimum 1

Binary indexed tree or segment tree?

	Binary indexed tree	**Segment tree**
Operations	sums (invertible operations)	any associative operation
Memory	n values	$2n$ values
Code	$\approx$ 10 lines	$\approx$ 20 lines
Range updates	difference array (point queries)	lazy propagation (range queries too)
Constant factor	smaller	larger

References

- A. Laaksonen. *Competitive Programmer's Handbook* (draft, July 2018), Chapters 9 and 28.
- P. M. Fenwick. A new data structure for cumulative frequency tables. *Software: Practice and Experience* 24(3), 1994.
- M. A. Bender, M. Farach-Colton. The LCA problem revisited. *LATIN* 2000.
- CSES Problem Set, `cses.fi/problemset`.